

UNIVERSIDAD TECNOLÓGICA DE PEREIRA

PROGRAMA DE INGENIERÍA DE ELECTRÓNICA / PROGRAMACIÓN II

Diseño de Retos Interactivos

Módulo: Tipos de Datos, Operadores y Control de Flujo (Python Quest)

Presentado por: Luis Mario Vélez

Fecha de entrega: 18 de septiembre de 2026

Docente: Juan Pablo Tabares

SECCIÓN CLASE 1

Tema: Tipos de Datos, Variables y Estructuras en Python

Métrica / Enfoque: Modificación de código base, selección en línea y ordenamiento de lógica.

Reto 1: RELLENAR ESPACIOS (Formateo con f-strings y Tipos)

- Mecánica: Completar los espacios en blanco (Inline Select / Fill-in-the-blank).
- Enunciado: En la Fábrica Alpina, se requiere generar un mensaje de alerta automático sobre la temperatura de los tanques. Selecciona las opciones correctas para formatear el mensaje mediante f-strings.
- Código Incompleto:

```
Python
tanque = "Leche M-01"
temperatura = 4.5 # float

# Completa la línea para interpolar las variables
mensaje = ____"El tanque {tanque} tiene una temperatura de {temperatura}°C"
print(____(temperatura)) # Debe mostrar <class 'float'>
```

- Opciones de Relleno:

1. Espacio 1: f | str | format
 2. Espacio 2: type | typeof | class
- Solución Válida:
 1. f
 2. type

Reto 2: MODIFICAR / REFACTORIZAR (Asignación Aumentada y Mutabilidad)

- Mecánica: Modificar una estructura existente para corregir su comportamiento.
- Enunciado: Un desarrollador junior creó la siguiente lista para registrar las frutas en inventario de un e-commerce, pero la fruta en la posición 1 cambió de "pera" a "mango" y el stock subió en 5 unidades. ¿Cuál es la forma pythónica de actualizar estos datos directamente?
- Código Predeterminado (Base):

```
frutas = ["manzana", "pera", "uva"]  
stock = 10
```

- Opciones de Modificación:
 - A) ,(Correcta)*

```
frutas[1] = "mango"  
stock += 5
```

- B)

```
frutas("pera") = "mango"  
stock = stock + "5"
```

- C)

```
frutas[2] = "mango"  
stock =+ 5
```

- Retroalimentación Académica:
 - Correcta: Las listas son mutables y sus índices inician en 0 (posición 1 es "pera"). El operador += realiza una asignación aumentada correcta.

Reto 3: ORDENAR / ARRASTRAR (Estructuración de Diccionarios y Pertenencia)

- Mecánica: Drag & Drop / Reorganizar bloques de código.
- Enunciado: En la fintech TuCuenta, necesitamos verificar si un cliente tiene la clave "ciudad" registrada en su perfil. Ordena las líneas para definir el diccionario y validar la pertenencia con el operador in.

- Bloques Desordenados:

1. print("ciudad" in cliente)
2. "nombre": "Lucas",
3. cliente = {
4. }
5. "edad": 20

- Secuencia Correcta Esperada:

```
cliente = {
    "nombre": "Lucas",
    "edad": 20
}
print("ciudad" in cliente)
```

Reto 4: DEPURA / CORRIGE (Inmutabilidad de Tuplas vs Listas)

- Mecánica: Identificar y corregir errores de tipo o sintaxis.
- Enunciado: El siguiente código intenta actualizar las coordenadas de entrega de un paquete, pero lanza un error TypeError: 'tuple' object does not support item assignment. ¿Por qué ocurre esto y cómo se debe solucionar?

- Código con Error:

- Opciones de Corrección:

- A) Cambiar los paréntesis () por corchetes [] para convertir coordenadas en una Lista (mutable).
- B) Usar el operador += para sumarle la nueva coordenada a la tupla.
- C) Reemplazar los datos usando la función type().
- Respuesta Correcta: A (Las tuplas son inmutables; si se requiere modificar sus elementos, se debe emplear una lista []).

Reto 5: SELECCIÓN RÁPIDA (Conjuntos y Eliminación de Duplicados)

- Mecánica: Identificación de propiedades de estructuras.
- Enunciado: Recibes una lista con IDs de transacciones repetidas: [101, 102, 101, 103, 102]. ¿Qué tipo de dato compuesto en Python debes utilizar si deseas conservar únicamente los elementos únicos sin duplicados?
- Opciones:
 - A) Lista (list)
 - B) Tupla (tuple)
 - C) Conjunto (set) (Correcta)
 - D) Diccionario (dict)

SECCIÓN CLASE 2

Tema: Operadores, Condicionales, Métodos y Entrada de Datos en Python

Métrica / Enfoque: Aplicación de contexto industrial/empresarial, control de flujo, validación de tipos e higiene de cadenas.

Reto 1: RELLENAR ESPACIOS (Entrada de datos y Métodos de Cadenas)

- Mecánica: Completar los espacios en blanco (Inline Select / Fill-in-the-blank).
- Enunciado: En el sistema de facturación de la planta de producción, se recibe la cantidad de unidades como un texto mediante input(). Debes limpiar los espacios accidentales del usuario, convertir la entrada a número entero y formatear el código de lote.
- Código Incompleto:

```
entrada_usuario = " 150 "
```

```
# 1. Elimina los espacios de los extremos
unidades_limpias = entrada_usuario.____()
```

```
# 2. Convierte el texto a entero para poder operar
unidades = ____ (unidades_limpias)
```

```
# 3. Formatea el lote "42" con ceros a la izquierda hasta 6 dígitos
codigo_lote = "42".____(6) # Resultado: "000042"
```

- Opciones de Relleno:
 1. Espacio 1: strip | lower | clear
 2. Espacio 2: int | float | str
 3. Espacio 3: zfill | center | format
- Solución Válida:
 1. strip
 2. int
 3. zfill

Reto 2: MODIFICAR / REFACTORIZAR (Operadores de División y Módulo)

- Mecánica: Modificar el código base para resolver una necesidad logística.
- Enunciado: La embotelladora Andina empaca gaseosas en cajas de 6 unidades. Dado un lote total de botellas, el sistema debe calcular cuántas cajas completas se arman y cuántas botellas quedan sueltas. El código actual usa una división incorrecta. Modifícalo utilizando los operadores de división entera (//) y módulo (%).
- Código Predeterminado (Base):

```
botellas_totales = 29
tamano_caja = 6

# CORRECCIÓN REQUERIDA EN ESTAS LÍNEAS:
cajas_completas = botellas_totales / tamano_caja # Produce decimales float (4.8333)
botellas_sobrantes = botellas_totales - tamano_caja
```

- Opciones de Modificación:
 - A) *(Correcta)*

```
cajas_completas = botellas_totales // tamano_caja
botellas_sobrantes = botellas_totales % tamano_caja
```

- B)

```
cajas_completas = botellas_totales % tamano_caja
botellas_sobrantes = botellas_totales // tamano_caja
```

- C)

```
cajas_completas = int(botellas_totales / tamano_caja)
botellas_sobrantes = botellas_totales ** tamano_caja
```

- Retroalimentación Académica:
 - Correcta: El operador // descarta los decimales devolviendo el número exacto de grupos completos (4 cajas), mientras que el operador % calcula el residuo exacto (5 botellas sobrantes).

Reto 3: ORDENAR / ARRASTRAR (Estructura Condicional elif y Operadores Lógicos)

- Mecánica: Drag & Drop / Reorganizar bloques de código (Parsons Problems).
- Enunciado: El departamento de Calidad evalúa el margen de error en piezas de ensamble. Reorganiza las siguientes líneas para construir la lógica que evalúa si la pieza es "Aprobada", "Bajo Revisión" o "Rechazada".
- Bloques Desordenados:
 1. else:
 2. elif margen_error <= 5.0:
 3. if margen_error <= 1.5:
 4. print("Estado: Aprobado")
 5. print("Estado: Bajo Revisión")
 6. print("Estado: Rechazado")

- Secuencia Correcta Esperada:

```
if margen_error <= 1.5:
    print("Estado: Aprobado")
elif margen_error <= 5.0:
    print("Estado: Bajo Revisión")
else:
    print("Estado: Rechazado")
```

Reto 4: DEPURA / CORRIGE (Operadores de Métodos de Lista vs. Diccionarios)

Mecánica: Identificar el método incorrecto y aplicar el ajuste adecuado.

Enunciado: El backend de la farmacéutica ejecuta el siguiente script para consultar el stock de un medicamento y agregar una nueva ubicación en la lista de almacenamiento. Sin embargo, el código arroja dos errores de ejecución (`AttributeError` y `KeyError`). ¿Cómo se debe corregir?

Código con Error:

```
almacen = {"bodega_1": 450, "bodega_2": 120}  
pasillos = ["A1", "B2"]
```

ERROR 1: Uso erróneo del método para consultar claves sin fallar
stock = almacen["bodega_3"]

ERROR 2: Intenta usar un método de lista que no existe para añadir
pasillos.update("C3")

- Opciones de Corrección:
 - A) Cambiar almacen["bodega_3"] por almacen.get("bodega_3", 0) y reemplazar .update() por .append("C3"). (Correcta)
 - B) Cambiar almacen["bodega_3"] por almacen.pop("bodega_3") y reemplazar .update() por .insert("C3").
 - C) Usar almacen.keys("bodega_3") y reemplazar .update() por .extend("C3").
- Respuesta Correcta: A (get() evita el colapso del programa si la clave no existe devolviendo un valor por defecto, mientras que .append() es el método adecuado para añadir un elemento individual al final de una lista).

Reto 5: SELECCIÓN RÁPIDA (Comprobación de Formatos con Métodos Str)

- Mecánica: Identificación de métodos booleanos de cadenas.
- Enunciado: Un formulario de registro de la plataforma exige que el número de cédula del usuario contenga únicamente dígitos numéricos (sin espacios, ni letras, ni guiones). ¿Cuál método de las cadenas de texto (str) permite validar esta condición retornando True o False?
- Opciones:
 - A) .isalpha()
 - B) .isdigit() (Correcta)
 - C) .isalnum()
 - D) .isspace()

SECCIÓN CLASE 4

Tema: Métodos de Cadenas de Texto (str) y Listas (list) en Python

Objetivo de Aprendizaje: Manipulación, transformación, búsqueda y ordenamiento de secuencias mediante métodos nativos en contextos de ingeniería.

Ejercicio 4.1: Completamiento de código – Métodos de Limpieza y Formato de Cadenas

- Mecánica: Completar espacios en blanco (Fill-in-the-blank).
- Enunciado: El sistema de recepción de datos de un sensor industrial recibe códigos de trama con espacios sobrantes y texto en minúsculas. Se requiere limpiar los extremos, convertir el código a mayúsculas y validar si finaliza con la extensión .RAW.
- Código Incompleto:

```
trama_entrada = " sensor_temp_01.raw "
```

1. Elimina espacios en blanco de ambos extremos
trama_limpia = trama_entrada.__()

2. Convierte el texto resultante a mayúsculas
trama_mayus = trama_limpia.__()

3. Comprueba si finaliza con la extensión requerida
es_valido = trama_mayus.__(".RAW")

- Opciones de Relleno:
 1. Espacio 1: strip | replace | split
 2. Espacio 2: upper | capitalize | title
 3. Espacio 3: endswith | startswith | find
- Solución Válida:
 1. strip
 2. upper
 3. endswith

Ejercicio 4.2: Refactorización – Inserción y Modificación de Listas Industriales

- Mecánica: Modificación de código base (Refactoring).
- Enunciado: Una línea de ensamblaje gestiona los IDs de productos en una lista. Se requiere insertar una pieza prioritaria en la segunda posición (índice 1) y extender la lista principal añadiendo una nueva tanda de componentes sin crear sublistas anidadas.
- Código Predeterminado (Base)

```
# CÓDIGO CON ERRORES DE MÉTODOS
piezas = ["P-10", "P-30"]
nuevas_piezas = ["P-40", "P-50"]
```



```
# ERROR: Están usando append para una lista completa y no ubican el índice correcto
piezas.append(1, "P-20")
piezas.append(nuevas_piezas)
```

- Opciones de Modificación:

- A)*(Correcta)*

```
piezas.insert(1, "P-20")
piezas.extend(nuevas_piezas)
```

- B)

```
piezas.add(1, "P-20")
piezas.push(nuevas_piezas)
```

- C)

```
piezas.replace(1, "P-20")
piezas.update(nuevas_piezas)
```

- Retroalimentación Académica:

- Correcta: insert(1, "P-20") ubica el elemento en el índice específico, mientras que extend() incorpora los elementos de la colección de manera individual (a diferencia de append(), que insertaría la lista entera como un único elemento anidado).

Ejercicio 4.3: Depuración – Ordenamiento Avanzado con el Parámetro key

- Mecánica: Identificar errores de sintaxis y lógica en métodos de listas.
- Enunciado: Un ingeniero busca ordenar una lista de lecturas de texto por su longitud utilizando el método .sort(). Sin embargo, cometió un error al invocar la función de criterio de ordenamiento.
- Código con Error:

```
lecturas = ["SensorA", "S1", "SensorPrincipal", "S_Temp"]
```

```
# ERROR: Se está ejecutando la función en lugar de pasar la referencia
```

```
lecturas.sort(key=len())  
print(lecturas)
```

Opciones de Corrección:

- A) Cambiar `key=len()` por `key=len` [cite: 4]. *(Correcta)*
- B) Reemplazar `sort()` por `sorted()` y agregar `reverse=None`
- C) Escribir `key="len"`.

Respuesta Correcta: A (Se debe proporcionar la función `len` como referencia para que Python la ejecute internamente sobre cada elemento; no se deben incluir paréntesis `()`)

SECCIÓN CLASE 5

Tema: Métodos de Conjuntos (`set`) y Diccionarios (`dict`) en Python

Objetivo de Aprendizaje: Operaciones de teoría de conjuntos, gestión de claves-valores y acceso seguro a estructuras compuestas

Ejercicio 5.1: Selección Rápida – Acceso Seguro a Diccionarios con `.get()`

Mecánica: Selección única sobre manejo de exenciones/métodos seguros.

Enunciado: Al consultar el estado de un componente en un diccionario de configuración, se desea evitar que el programa lance un error de tipo `KeyError` si la clave no existe. ¿Qué método permite definir un valor de retorno por defecto si la clave no se encuentra?

Opciones:

- A) `diccionario.find("estado", "Desconocido")`
- B) `diccionario.get("estado", "Desconocido")` *(Correcta)*
- C) `diccionario.popitem("estado")`
- D) `diccionario.setdefault()` sin parámetros

Ejercicio 5.2: Ordenamiento de Bloques – Comparación de Conjuntos (Parsons Problem)

Mecánica: Reorganizar bloques de código (*Drag & Drop*).

Enunciado: Se tienen dos conjuntos de dispositivos detectados por dos antenas en la planta (`antena_A` y `antena_B`). Reorganiza las líneas de código para encontrar los dispositivos comunes entre ambas antenas (intersección) y los dispositivos exclusivos de la Antena A (diferencia) [cite: 4].

* **Bloques Desordenados:** *

1. `exclusivos_A = antena_A.difference(antena_B)`
2. `antena_A = {"DEV-1", "DEV-2", "DEV-3"}`
3. `comunes = antena_A.intersection(antena_B)`
4. `antena_B = {"DEV-2", "DEV-3", "DEV-4"}`

Secuencia Correcta Esperada:

```
antena_A = {"DEV-1", "DEV-2", "DEV-3"}
antena_B = {"DEV-2", "DEV-3", "DEV-4"}

comunes = antena_A.intersection(antena_B)
exclusivos_A = antena_A.difference(antena_B)
```

SECCIÓN CLASE 6

Tema: Funciones integradas (Built-in Functions) y de Conversión en Python

Objetivo de Aprendizaje: Aplicar funciones nativas de Python para análisis numérico, manipulación de colecciones, conversión de tipos de datos y control de flujo en entornos informáticos e industriales.

Ejercicio 6.1: Selección Rápida – Conversión de Tipos de Datos e Ingreso de Datos

- Mecánica: Selección única sobre casting y tipos de datos en la recepción de entradas.
- Enunciado: Un ingeniero necesita ingresar la corriente tomada por un motor eléctrico a través de la consola mediante `input()` y calcular la potencia sumándole una constante. ¿Por qué el código `lectura = input("Corriente: ")` requiere envolverse con la función `float()` para operar matemáticamente?
- Opciones:
 - A) Porque `input()` devuelve por defecto un entero (`int`) y no permite operaciones decimales.
 - B) Porque `input()` siempre retorna una cadena de texto (`str`), impidiendo realizar operaciones aritméticas directas. (Correcta)
 - C) Porque la función `float()` permite limpiar los espacios en blanco de la lectura automáticamente.
 - D) Porque `input()` genera un booleano si se ingresan números.
- Respuesta Correcta: B (La función `input()` retorna siempre un valor tipo `str`, por lo que para procesarlo numéricamente se debe usar conversión explícita mediante `float()` o `int()`).

Ejercicio 6.2: Completamiento de código – Procesamiento de Métricas e Promedios

- Mecánica: Completar espacios en blanco (Fill-in-the-blank).
- Enunciado: Se han registrado las lecturas de voltaje de una batería durante un ciclo de carga (voltajes = [12.1, 12.8, 13.5, 12.4]). Se requiere obtener la cantidad de muestras, el valor pico máximo y calcular el voltaje promedio redondeado a dos decimales.
- Código Incompleto:

```
voltajes = [12.1, 12.8, 13.5, 12.4]
```

```
# 1. Obtener la cantidad total de muestras recibidas
```

```
total_muestras = ____ (voltajes)
```

```
# 2. Obtener el voltaje máximo del ciclo
```

```
v_max = ____ (voltajes)
```

```
# 3. Calcular el promedio y redondearlo a 2 decimales
```

```
promedio = sum(voltajes) / total_muestras
```

```
promedio_final = ____ (promedio, 2)
```

- Opciones de Relleno:
 1. Espacio 1: len | count | size
 2. Espacio 2: max | sum | abs
 3. Espacio 3: round | int | pow
- Solución Válida:
 1. len
 2. max
 3. round

Ejercicio 6.3: Refactorización – Mapeo e Iteración Simultánea de Señales

- Mecánica: Modificación de código base (Refactoring).
- Enunciado: Un sistema adquiere nombres de canales de adquisición de datos y sus frecuencias de muestreo asociadas. Se requiere recorrer ambas listas en paralelo utilizando la función integrada zip() e indexar el orden con enumerate().
- Código Predeterminado (Base):

```
# CÓDIGO INEFICIENTE QUE USA ÍNDICES MANUALES
```

```
canales = ["Sensor_A", "Sensor_B", "Sensor_C"]
```

```
frecuencias = [1000, 2000, 1500]
```

```
for i in range(len(canales)):
```

```
    print(f"Canal {i+1}: {canales[i]} opera a {frecuencias[i]} Hz")
```

- Opciones de Modificación (Refactorización elegante):
 - A)*(Correcta)*

```
for idx, (canal, freq) in enumerate(zip(canales, frecuencias), start=1):  
    print(f"Canal {idx}: {canal} opera a {freq} Hz")
```

- B)

```
for idx, canal, freq in range(zip(canales, frecuencias)):  
    print(f"Canal {idx}: {canal} opera a {freq} Hz")
```

- C)

```
for idx, (canal, freq) in map(canales, frecuencias):  
    print(f"Canal {idx}: {canal} opera a {freq} Hz")
```

- Retroalimentación Académica:
 - Correcta: zip(canales, frecuencias) combina ambas colecciones elemento a elemento en tuplas, mientras que enumerate(..., start=1) genera el índice numérico iniciando de forma limpia desde 1 para presentar los datos al operador.

Ejercicio 6.4: Depuración – Evaluación Booleana de Rangos con all() y any()

- Mecánica: Identificar errores de lógica/sintaxis en funciones booleanas.
- Enunciado: Se desea comprobar si todas las temperaturas registradas en un rack de servidores están dentro del rango seguro (menores a 50°C). El programador intentó usar la función integrada all() pero cometió un error de sintaxis en la expresión generadora.
- Código con Error:

```
temperaturas = [42.5, 38.0, 48.2, 51.1]
```

```
# ERROR DE SINTAXIS: Uso incorrecto de la expresión dentro de all()  
alerta = all(temperaturas < 50)  
print(f"¿Estado seguro?: {alerta}")
```

Opciones de Corrección:

- A) Reemplazar por: `alerta = all(t < 50 for t in temperaturas)` *(Correcta)*
- B) Reemplazar por: `alerta = temperaturas.all(50)`
- C) Reemplazar por: `alerta = any(temperaturas) < 50`

Respuesta Correcta:A (Las funciones ``all()`` y ``any()`` requieren recibir un iterable o una expresión generadora que evalúe cada elemento de forma individual ``t < 50 for t in temperaturas``).

Ejercicio 6.5: Análisis Técnico – Diferencia Fundamental entre Función Integrada y Método Mecánica: Clasificación conceptual sobre sintaxis de Python.

Enunciado: Determina la diferencia estructural al invocar una función integrada frente a un método en Python.

Opciones:

- A) Las funciones integradas se ejecutan pasando el objeto como argumento (ej. ``len(lista)``), mientras que los métodos pertenecen a la clase del objeto y se invocan usando la notación de punto (ej. ``lista.append(val)``). *(Correcta)*
- B) Los métodos solo funcionan con datos numéricos y las funciones integradas solo con cadenas de texto.
- C) Las funciones integradas modifican el objeto original en memoria, mientras que los métodos devuelven siempre ``None``.